

Termal Görüntülerde Küçük Hava Hedeflerinin Kalman Filtresi ve Yeniden Yakalama ile Kilitlenmeli Takibi

İngilizce ad: Lock-On Tracking of Small Aerial Targets in Thermal Imagery with Kalman Filtering and Re-Acquisition **Repo adı:** `thermal-uav-lockon-tracker`

Savunma sanayii odaklı, görüntü işleme + makine öğrenmesi projesi. Süre: ~3 hafta, haftada 20-40 saat. Dil: Python. Hedef: Çalışan bir demo + algoritma analizi içeren teknik rapor.

0. Bu proje tam olarak ne?

Termal (IR) bir video akışında küçük bir hava hedefini (drone/uçak) **tespit eden**, ona **kilitlenen**, hedef birkaç kare kaybolursa bile pozisyonunu **tahmin ederek takibi sürdüren**, ve birden çok hedef varsa **tek bir hedefe odaklanan** bir sistem.

Bu, gerçek bir EO/IR güdüm / atış kontrol sisteminin yazılım çekirdeğinin basitleştirilmiş halidir. Donanım yok; her şey hazır video verisi üstünde çalışacak.

Başarı kriteri (proje sonunda elinde olması gerekenler)

- Termal video üstünde hedefi kare kare takip eden, kilit kutusu çizen bir demo videosu.
- "Sadece tespit" (baseline) ile "tespit + takip" yönteminin **sayısal karşılaştırması** (grafiklerle).
- Her algoritmanın *neden* öyle çalıştığını anlatan 4-6 sayfalık teknik rapor.
- Temiz, README'li bir GitHub reposu.

Neyi yapmayacaksın (kapsam dışı — vakit kaybetme)

- Gerçek donanım, gerçek kamera, gerçek drone.
- SLAM, navigasyon, gerçek RF/güdüm fiziği.
- Sıfırdan derin öğrenme mimarisi yazmak (hazır YOLO kullanacaksın).

1. Kavramsal temel (kod yazmadan önce oku)

Bu bölümü anlamadan koda geçme. Mülakatta sorulacak olan şeyler bunlar.

1.1. Tespit (detection) vs takip (tracking) farkı

- Tespit:** Tek bir karede "burada bir hedef var" demek. Her kare bağımsız. Hedef bir kare görünmezse o karede hiçbir şey yok.
- Takip:** Kareler arasında aynı hedefi **bağlamak**. "Önceki karedeki hedef = bu

karedeki şu kutu" demek, ve hedef kaybolrsa bile nerede olduğunu **hatırlamak/tahmin etmek**.

Saf tespitçi titrek ve unutkanlıktır. Takip katmanını sürekliliği ve hafızayı ekler.

1.2. Tracking-by-detection paradigması

En yaygın ve en pratik yaklaşım:

1. Her karede tespit çalıştır → kutular elde et.
2. Bu kutuları mevcut "iz"lerle (track) eşleştir (data association).
3. Eşleşen izleri güncelle, eşleşmeyenler için tahmin yürüt.
4. Yeni tespitler için yeni iz başlat, çok uzun kaybolanları sil.

1.3. Kalman Filtresi — sezgi

Hedefin **durumunu** tut: $[x, y, vx, vy]$ (konum + hız). İki adımı vardır:

- **Predict (tahmin)**: Sabit hız varsayımıyla bir sonraki konumu tahmin et. Tespit gelmese bile bu adım çalışır → işte "kaybolan hedefi tahmin etme" yeteneği buradan gelir.
- **Update (düzeltme)**: Tespit gelince, ölçümle tahmini birleştirip durumu düzelt.

Neden işe yarar? Gürültülü, ara sıra kaybolan ölçümlerden pürüzsüz bir yörünge çıkarır. Mülakat sorusu: "Tespit bir kare ıskalandığında ne olur?" Cevap: "Sadece predict adımı çalışır, iz tahminle ilerler."

1.4. Data association — sezgi

İki hedef varsa, yeni iki tespit hangi ize ait? Bu bir **atama (assignment)** problemidir. Maliyet matrisi kurarsın (her iz-tespit çiftinin uzaklığı/IoU'su), sonra **Hungarian algoritması** ile toplam maliyeti minimize eden eşleşmeyi bulursun. Tek hedefte basit "en yakın komşu + gating (eşik)" yeter.

1.5. Durum makinesi (state machine) — projenin kalbi

Kilit dört durumda gezer:

- **Aranıyor**: Hedef yok, taranıyor. Tespit gelirse → Kilitli.
- **Kilitli**: Tespit sürüyor, kutu güncelleniyor. Tespit ıskalanırsa → Tahmin.
- **Tahmin (coast)**: Tespit yok, Kalman tahmin ediyor. Yeniden tespit gelirse → Kilitli. N kare boyunca gelmezse → Kayıp.
- **Kayıp**: Kilit bırakıldı, takip sıfırlanır → Aranıyor.

Bu makineyi düzgün kurman, projenin "saf akademik tracker" değil "savunma sistemi" gibi görünmesini sağlar.

1.6. Lock-on policy (kilitlenme politikası)

Çoklu hedef takibi tüm izleri tutar; ama gerçek bir güdüm sistemi **tek** hedefe kilitlenir. Bir politika katmanı seçer: en yakın / en büyük / en yüksek skorlu izi seç, onu sürekli raporla, kaybolursa yeniden yakala. Bu katman olmadan proje yarım kalır.

1.7. Küçük hedef problemi

Termalde hedef genelde 5-15 piksellik bir nokta. Standart YOLO küçük nesnelerde zayıftır. Çözümler:

- Giriş çözünürlüğünü artır (imgsz büyüt).
- **Tiling / slicing inference**: Görüntüyü parçalara böl, her parçada tespit yap, sonuçları birleştir (SAHI kütüphanesi). Küçük hedeflerde belirgin iyileşme sağlar.

2. Ortam kurulumu

2.1. Gereksinimler

- Python 3.10+
- (Opsiyonel ama iyi) NVIDIA GPU. Yoksa CPU'da da çalışır, sadece eğitim/çıkarım yavaşlar.

2.2. Sanal ortam ve kütüphaneler

```
bash

python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate

pip install ultralytics          # YOLO (tespit)
pip install opencv-python        # video, görselleştirme
pip install numpy scipy          # matematik, Hungarian (linear_sum_assignment)
pip install filterpy             # hazır Kalman (öğrenmek için kendini de y
pip install sahi                 # küçük hedef için slicing inference
pip install matplotlib           # grafikler
pip install motmetrics           # MOTA/IDF1 takip metrikleri
```

2.3. Önerilen repo yapısı

```
termal-hedef-takip/
├── README.md
├── requirements.txt
├── data/                # veri seti (git'e koyma, .gitignore'a ekle)
└── src/
```

```
| | └─ detector.py          # YOLO + SAHI sarmalayıcı
| | └─ kalman.py           # Kalman filtresi (kendi implementasyonun)
| | └─ association.py      # Hungarian / nearest-neighbor eşleştirme
| | └─ tracker.py         # iz yönetimi + durum makinesi + lock-on
| | └─ evaluate.py        # metrikler, baseline karşılaştırması
| | └─ visualize.py       # video üstüne çizim
| └─ experiments/        # deney scriptleri ve sonuç grafikleri
| └─ report/             # teknik rapor (md veya pdf)
| └─ demo/               # çıktı demo videoları
```

3. Veri seti

3.1. Birincil: Anti-UAV

- Termal + görünür video dizileri, üstünde drone'lar, kutu etiketleriyle.
- Tespit ve takip için tasarlanmış → tam senin senaryon.
- "Anti-UAV dataset" diye ara; akademik challenge veri setidir, açık erişimli sürümleri var.

3.2. İkincil (opsiyonel): FLIR ADAS

- Genel termal nesne veri seti (yaya, araç). Termal görüntü işleme alıştırmaları için iyi.

3.3. Veri hazırlık adımları

1. Birkaç video dizisini indir, klasör yapısını anla.
 2. Etiket formatını incele (genelde her kare için kutu koordinatları).
 3. YOLO eğitimi yapacaksan etiketleri YOLO formatına çevir (`(class x_center y_center w h)`, normalize).
 4. **Kısayol:** Eğitim yerine, hazır bir COCO-önceden-eğitilmiş YOLO ile başlayıp termalde nasıl çalıştığını gör. İyi değilse, az sayıda etiketli kareyle **fine-tuning** yap (transfer learning). 3 haftada tam eğitim şart değil; fine-tuning yeter.
-

4. HAFTA 1 — Tespit pipeline'ı

Hedef: Hafta sonunda termal video üstünde kare kare kutu çizebiliyor olmak.

Adım 1.1 — Veriyi yükle ve göster

OpenCV ile video/kare dizisini aç, ekranda göster. Etiketli kutuları (ground truth) çiz.

Amaç: veriyi gözünle anlamak.

Adım 1.2 — Temel YOLO testini

Adım 1.2 — Temel YOLO tespiti

python

```
from ultralytics import YOLO
model = YOLO("yolov8n.pt")          # küçük model, hızlı
results = model(frame)              # tek kare
boxes = results[0].boxes.xyxy.cpu().numpy() # [x1,y1,x2,y2]
```

Termalde nasıl çalıştığını gözle. Muhtemelen küçük hedefleri kaçıırır — bu beklenen, sonraki adımda düzelteceğiz.

Adım 1.3 — Fine-tuning (gerekirse)

Az sayıda etiketli termal kareyle modeli ince ayarla:

python

```
model.train(data="antiuav.yaml", epochs=50, imgsz=1280)
```

`imgsz`'yi büyütmek küçük hedefte yardımcı olur.

Adım 1.4 — Küçük hedef için SAHI (slicing)

python

```
from sahi import AutoDetectionModel
from sahi.predict import get_sliced_prediction
det_model = AutoDetectionModel.from_pretrained(
    model_type="ultralytics", model_path="best.pt", confidence_threshold=0.3
)
result = get_sliced_prediction(frame, det_model,
    slice_height=512, slice_width=512,
    overlap_height_ratio=0.2, overlap_width_ratio=0.2)
```

SAHI öncesi/sonrası recall'u karşılaştır → raporda ilk grafiğin.

Hafta 1 çıktısı

`detector.py`: kare alır, kutu listesi döndürür. Bir video üstünde çalışan, kutu çizen demo.

5. HAFTA 2 — Takip, durum makinesi, kilitlenme

Hedef: Hafta sonunda hedefe kilitlenip, kaybolduğunda tahminle takibi sürdüren sistem.

Adım 2.1 — Kalman filtresini yaz

Durum: $[x, y, vx, vy]$. Sabit hız modeli. `filterpy` ile başlayabilirsin ama **matematiğini anla** (predict ve update denklemleri). Kendi mini implementasyonunu yazmak rapor için çok değerli.

python

```
import numpy as np
class KalmanTracker:
    def __init__(self, x, y):
        self.state = np.array([x, y, 0, 0], dtype=float) # x,y,vx,vy
        self.P = np.eye(4) * 10.0 # belirsizlik
        dt = 1.0
        self.F = np.array([[1,0,dt,0],[0,1,0,dt],[0,0,1,0],[0,0,0,1]]) # ge
        self.H = np.array([[1,0,0,0],[0,1,0,0]]) # ölçüm modeli
        self.Q = np.eye(4) * 0.1 # süreç gürültü
        self.R = np.eye(2) * 1.0 # ölçüm gürültü

    def predict(self):
        self.state = self.F @ self.state
        self.P = self.F @ self.P @ self.F.T + self.Q
        return self.state[:2] # tahmini x,y

    def update(self, z): # z = ölçülen [
        y = z - self.H @ self.state # innovation
        S = self.H @ self.P @ self.H.T + self.R
        K = self.P @ self.H.T @ np.linalg.inv(S) # Kalman kazanc
        self.state = self.state + K @ y
        self.P = (np.eye(4) - K @ self.H) @ self.P
```

Adım 2.2 — Data association

Her karede: izlerin tahmini konumları ile yeni tespitler arasında maliyet matrisi (uzaklık ya da 1-IoU) kur, Hungarian ile eşleştir.

python

```
from scipy.optimize import linear_sum_assignment
cost = np.linalg.norm(track_preds[:,None,:] - dets[None,:,:], axis=2)
row, col = linear_sum_assignment(cost)
# gating: cost[r,c] çok büyükse eşleşmeyi reddet (kabul edme)
```

Adım 2.3 — İz yönetimi + durum makinesi

Her iz için bir Kalman + bir durum + "kaç kare ıskalandı" sayacı tut.

python

```
# her kare:
```

```
# 1) tüm izler için predict()
# 2) tespitleri izlerle eşleştir (association)
# 3) eşleşen iz: update(), miss_count=0, durum=KILITLI
# 4) eşleşmeyen iz: miss_count+=1, durum=TAHMIN
#     miss_count > N ise durum=KAYIP, izi sil
# 5) eşleşmeyen tespit: yeni iz başlat (durum=ARANIYOR->KILITLI)
```

N (tolerans) bir hiperparametre — küçükse çabuk kopar, büyükse hayalet izler kalır. Raporda bunu tartış.

Adım 2.4 — Lock-on policy

Birden çok iz varsa tek birini seç (en yakın merkeze / en büyük / en yüksek güven). Seçilen izi "kilitli hedef" olarak sürekli raporla. Kaybolursa yeniden yakalama mantığını ekle.

Hafta 2 çıktısı

`tracker.py`: video üstünde tek hedefe kilitlenip onu takip ediyor; hedef kısa süre kaybolunca kutu tahminle devam ediyor (farklı renkle göster: kilitli=yeşil, tahmin=sarı).

6. HAFTA 3 — Deney, ölçüm, rapor

Hedef: Yöntemin işe yaradığını **kanıtlamak** ve yazmak.

Adım 3.1 — Baseline kur

Aynı tespitçiyi al, **takip katmanını çıkar** (her kare bağımsız, hafıza yok). Bu senin karşılaştırma noktan.

Adım 3.2 — Metrikler

- **Tespit**: precision, recall (SAHI'li / SAHI'siz).
- **Takip**: `motmetrics` ile MOTA (genel doğruluk), IDF1 (kimlik tutarlılığı), ID switch sayısı (kilit kaç kez karıştı).
- **Hız**: FPS (gerçek zamanlı mı?).

Adım 3.3 — Kilit deneyi (occlusion)

Hedefi kasten K kare boyunca gizle (kutuları sil / kareyi karart). Ölç:

- Baseline: kilidi anında kaybeder.
- Senin sistemin: Kalman tahminiyle K kare dayanır, hedef geri gelince yeniden yakalar.

Bu **tek deney** projenin değerini kanıtlar. "Tahmin toleransı N" ile "kaç kare kaybı tolere edilebilir" eğrisini çıkar.

Adım 3.4 — Hiperparametre taraması

N (miss toleransı), gating eşiği, güven eşiği gibi parametreleri değiştir, metriklerin nasıl değiştiğini grafiklerle. Bu mühendislik olgunluğu gösterir.

Adım 3.5 — Rapor ve repo

Aşağıdaki rapor iskeletini doldur, demo videosu çek, README yaz.

7. Teknik rapor iskeleti (4-6 sayfa)

- Giriş / Problem:** GPS/iletişim kısıtlı savunma senaryosunda termal hava hedefi takibi neden önemli. Küçük hedef + termal + gerçek zaman zorlukları.
- İlgili çalışmalar (kısa):** tracking-by-detection, SORT/DeepSORT, küçük nesne tespiti.
- Yöntem:**
 - Tespit (YOLO + SAHI), neden slicing.
 - Kalman filtresi (durum modeli, predict/update denklemleri).
 - Data association (Hungarian, gating).
 - Durum makinesi (4 durum, geçişler) — diyagramını koy.
 - Lock-on policy.
- Deneyler:** veri seti, metrikler, baseline.
- Sonuçlar:** tablolar + grafikler (SAHI etkisi, occlusion dayanıklılığı, N taraması, FPS).
- Tartışma / Sınırlar:** nerede başarısız oluyor, gerçek sisteme geçişte ne eksik.
- Karmaşıklık analizi:** Kalman $O(1)$ /iz, association $O(n^3)$ (Hungarian) ya da küçük n için pratik maliyet, kare başına toplam maliyet.

İpucu: Daha önce çizdiğimiz **durum makinesi diyagramını** rapora koy. Görsel anlatımı çok güçlendirir.

8. Yapay zekayı doğru kullanma (önemli)

Bu projeyi AI yardımıyla yapacaksın. İşe alım değerini korumak için:

- Yazdır:** İskelet kod, görselleştirme, deney döngüsü, grafik kodu, README taslağı, veri dönüştürme. Bunlar zaman kazandırır.
- Anlamadan geçme:** Kalman denklemleri, Hungarian'ın neden çalıştığı, durum makinesi mantığı, karmaşıklık analizi. AI'a "bunu **bana açıkla**, neden böyle" diye sor; "yaz geç" deme.

- **Test sorusu:** Her parça için kendine sor — "Bunu mülakatta beyaz tahtada anlatabilir miyim?" Cevap hayırsa, o parçayı tekrar çalış.

9. Sık yapılan hatalar

- Tüm vakti tespite harcıyıp takibe (asıl katkıya) zaman kalmaması. → Tespiti "yeterince iyi" bırakıp Hafta 2'ye geç.
- Baseline koymamak. → Karşılaştırmaz sonuç değersizdir.
- Kalman'ı (filterpy)'dan çağırıp matematiğini öğrenmemek. → En az bir kez kendin yaz.
- Gating eşiğini ayarlamamak → izler birbirine karışır (ID switch patlar).
- README ve raporu sona bırakıp yetiştirememek. → Her hafta sonunda README'yi güncelle.

10. Kaynaklar (arama terimleri)

- "Anti-UAV dataset", "FLIR ADAS thermal dataset"
- "SORT simple online realtime tracking paper"
- "Kalman filter object tracking tutorial"
- "Hungarian algorithm data association"
- "SAHI slicing aided hyper inference small object"
- "py-motmetrics MOTA IDF1"
- "Ultralytics YOLO custom training"

11. Tek bakışta 3 haftalık özet

Hafta	Odak	Somut çıktı
1	Tespit (YOLO + SAHI, termal)	Video üstünde kare kare kutu çizen detektör
2	Takip (Kalman + association + durum makinesi + lock-on)	Hedefe kilitlenip kaybolunca tahminle süren takip
3	Deney + ölçüm + yazım	Baseline karşılaştırması, grafikler, rapor, demo, repo

Altın kural: "Bir baseline'ı yen, tradeoff'u ölç ve grafikle, neden öyle yaptığını yaz."

Bu üçü olduğunda proje seni aday mühendisliğe taşır.